GDU_Android_SDK_2.0.108

GDU Android SDK 2.0.108说明文档

GDU_Android_SDK说明文档

修改时间	修改内容	修改人
2026-10-12	a. 2.4.1 GDUFlightController 失联保护配置：新增 setConnectionFailSafeBehavior 重载（behavior, loseTime, callback）设置动作及失联触发时间 loseTime（累计失联时间固件固定 60 秒）；新增 getConnectionFailSafeState(callback) 获取动作与 loseTime；新增 ConnectionFailSafeState 数据类（behavior + loseTime，单位秒）；2.4.1.11 增加 ConnectionFailSafeBehavior 枚举说明（HOVER/GO_HOME）；2.4.1.12 增加 ConnectionFailSafeState 类说明；新增 setReturnToDockAlternateHeight(int height, callback) / getReturnToDockAlternateHeight(callback) 设置/获取飞往机场备降点的高度（单位 cm）	GDU
2026-10-09	a. 2.4.1 GDUFlightController 增加 setIMUStateCallback：IMU校准状态监听接口，回调返回校准状态、校准姿态与校准进度；addIMUCalibrationCallback 标记废弃 b. 2.4.1.10 增加 IMUState 类说明：CalibrationState（校准状态）、CalibrationOrientation（校准姿态）、getCalibrationProgress（校准进度）	GDU
2026-09-29		GDU

	a. 2.4.3.1 LTEManager 增加 setLTEPushMode/getLTEPushMode：LTE推流模式设置与获取接口	
2026-09-28	a. 2.4.5 GDUCamera 增加变焦焦距/倍数监听、测光模式、曝光模式、视频码流类型、对焦模式/目标/等接口 b. 增加枚举类说明：MeteringMode、ExposureMode、VideoStreamType、FocusMode	GDU
2026-09-08	a. 2.4.5 GDUCamera 增加红外伪彩设置和获取接口（setThermalPalette/getThermalPalette） b. 2.4.5 GDUCamera 增加红外测温接口：setThermalMeasurementMode/setThermalSpotMeteringTargetPoint/setThermalMeteringArea/setThermalTemperatureDataEnabled/getThermalTemperatureData c. 2.4.5.6 增加 CameraThermalPalette：红外伪彩枚举说明 d. 2.4.5.7 增加 ThermalTemperatureMeasureMode：红外测温模式枚举说明 e. 2.4.5.8 增加 TMSGimbalState：红外测温数据结构说明	GDU
2026-09-03	a. 2.5.2 WaypointMissionOperator 增加 pushKMZFileToAircraft：上传KMZ文件至飞机端接口 b. 2.5.2 增加 TaskType：航线文件上传任务类型枚举说明	GDU

添加GDU_SDK_ANDROID开发包

1.1.1、添加依赖文件

将无人机SDK 的 aar包和so文件复制到工程（此处截图以示例Demo为例子）的 libs 目录下，如果有老版本 aar 包在其中，请删除。如图所示

1.1.2、在主工程的build.gradle文件配置dependencies

（1）添加SDK依赖， dependencies 配置方式如下：

代码块

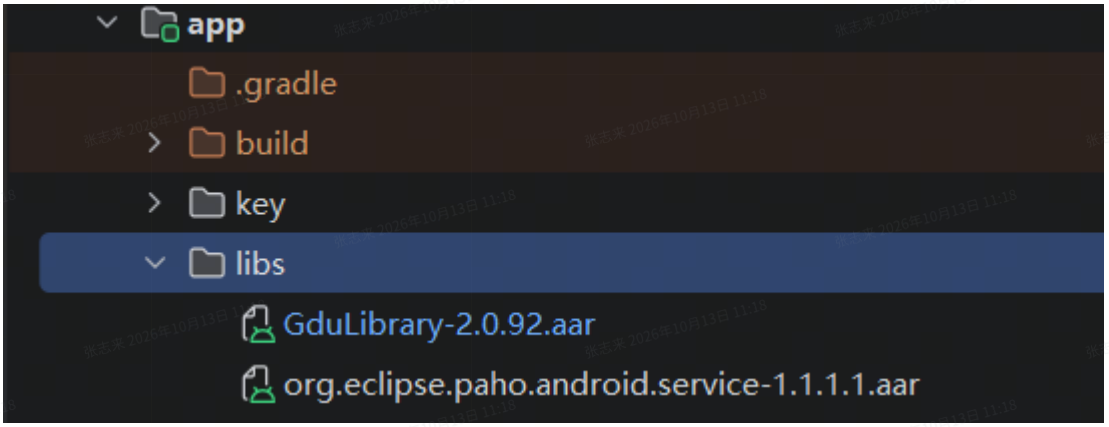
```
api fileTree(include: ['*.jar', '*.aar'], dir: 'libs')
```

（2）so配置到build.gradle的android中，方式如下：

代码块

```
sourceSets {
    main {
        jniLibs.srcDirs = ['libs']
    }
}
```

（3）主工程的build.gradle文件在Project目录中位置：



1.1.3、配置权限

在AndroidManifest.xml中配置权限:

代码块

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

接口参考文档

2.1、MANAGER CLASSES

2.1.1 GDUSDKManager

GDUSDKManager是将SDK与普宙飞行器一起使用的入口。这个类主要用于注册SDK，连接和访问飞行器。

限定符和类型	方法和说明
GDUSDKManager	**getInstance**() 获取GDUSDKManager实例
BaseProduct	**getProduct**() 获取当前飞行器实体类
BaseProduct.Model	getModel() 获取飞机类型
void	**registerApp**(Context context, SDKManagerCallback callback) 注册SDK
void	**startConnectionToProduct**() 开始建立SDK和飞行器的连接，这个方法必须要在registerApp成功后调用
void	**stopConnectionToProduct**() 断开SDK和飞行器的连接
void	**getSDKVersion** () 获取SDK的版本号
void	setCustomRCEnable(boolean isEnable) *设置自定义遥控器*
boolean	getCustomRCEnable() *获取是否为自定义遥控器*
void	setCustomLogPath(String logPath) *设置自定义日志路径（**需要在setLogEnable之前调用**）*
void	setLogEnable(boolean isEnable) *设置日志开关*

LiveStreamManager

LiveStremManager是将飞行器的视频流到RTMP服务器。

限定符和类型	方法和说明
LiveStremManager	**getInstance**() 获取LiveStremManager实例
void	**setLiveUrl**(String url) 设置直播的推流地址
String	**getLiveUrl** () 获取直播的推流地址
void	**startStream** () 开始直播推流
void	**pushLiveVideoDat**(byte[] videoData, int len) 推送H264/H265数据
void	**stopStream**() 停止直播推流
long	**getStartTime**() 获取直播的开始时间
boolean	**isStreaming**() 是否直播中
float	**getLiveVideoFps**() 获取视频的帧率
int	**getLiveVideoBitRate**() 获取视频流的码率
void	**addLiveErrorStatusListener**(OnLiveErrorStatusListener onLiveErrorStatusListener) 添加直播推流的状态监听
void	**removeLiveErrorStatusListener**(OnLiveErrorStatusListener onLiveErrorStatusListener) 移除直播推流的状态监听

2.1.2.1 OnLiveErrorStatusListener

OnLiveErrorStatusListener直播推流视频实时监听

限定符和类型	方法和说明
void	**onError**(int error, String description) 直播推流视频实时监听状态 error:错误码 description:错误描述

2.2、BASE CLASSES

2.2.1 BaseProduct

BaseProduct基础产品抽象类，飞行器将继承这个类，这个类将提供飞行器的连接，SN的获取，健康状态等功能

限定符和类型	方法和说明
void	**getProductSN**() 获取飞行器SN
Void	**setDiagnosticsInformationCallback**(GDUDiagnostics.DiagnosticsInformationCallback diagnosticsInformationCallback) 设置健康管理监听 GDUDiagnostics健康管理类
boolean	**isConnected**() 飞行器是否连接

2.2.1.1 VideoFeeder

VideoFeeder管理移动端实时流，获取相机流或者FPV流

限定符和类型	方法和说明
VideoFeeder.VideoFeed	**getPrimaryVideoFeed**() 获取主要的视频输入源，一般为相机输出

2.2.1.1.1 VideoFeed

VideoFeed视频源。用于接收来自相机源的视频数据。

限定符和类型	方法和说明
boolean	**addVideoDataListener**(VideoFeeder.VideoDataListener dataListener) 添加视频流数据监听 VideoDataListener: onReceive(byte[] datas, int len) datas: H264/H265数据 len: H264/H265数据长度

2.2.2 BaseComponent

BaseComponent基础组件抽象类，飞行器的各个组件将继承这个类，提供版本号获取，连接状态反馈等

限定符和类型	方法和说明
void	

	getFirmwareVersion(CommonCallbacks.CompletionCallbackWith<String> version) 获取组件的版本号
void	**setComponentListener**(BaseComponent.ComponentListener componentListener) 设置组件连接状态监听
boolean	**isConnected**() 组件是否连接

2.3、PRODUCT CLASSES

2.3.1 GDUAircraft

GDUAircraft飞行器实体类，包含飞行器基本信息，获取各个组件。实体类通过GDUSDKManager.getProduct()获取。

限定符和类型	方法和说明
GDUFlightController	**getFlightController**() 获取飞控组件
GDUBattery	**getBattery** () 获取电池组件
GDURemoteController	**getRemoteController** () 获取遥控器组件
BaseComponent	**getGimbal** () 获取云台组件
BaseComponent	**getCamera** () 获取相机组件
BaseComponent	**getAirLink**() 获取图传组件
BaseComponent	getLTE() 获取备份图传组件
BaseComponent	getNoFlyZone() 获取禁飞区
BaseComponent	getRadar() *获取雷达组件*
boolean	isConnected() 是否连接
void	setDiagnosticsInformationCallback(GDUDiagnostics.DiagnosticsInformationCallback diagnosticsInformationCallback) *健康管理监听*

2.4、COMPONENT CLASSES

2.4.1 GDUFlightController

GDUFlightController飞控组件，提供向飞行器发送各种控制指令。

限定符和类型	方法和说明
void	**startTakeoff**(CommonCallbacks.CompletionCallback callback) 发送起飞指令，飞机开始起飞，起飞到离地面相对高度5米悬停
void	startTakeoff(int height,CommonCallbacks.CompletionCallback callback) 一键起飞到设定高度悬停 单位cm
void	**cancelTakeoff**(CommonCallbacks.CompletionCallback callback) 停止飞行器起飞任务
void	**startLanding** (CommonCallbacks.CompletionCallback callback) 发送降落指令，飞行器从当前高度开始降落
void	**cancelLanding** (CommonCallbacks.CompletionCallback callback) 取消当前飞行器降落任务
void	**startGoHome** (CommonCallbacks.CompletionCallback callback) 发送返航指令，飞行已当前高度执行返航，如果飞行在起飞点上方则执行降落
void	**cancelGoHome** (CommonCallbacks.CompletionCallback callback) 取消当前返航任务
void	**startReturnToDock**(CommonCallbacks.CompletionCallback callback) *开始返航到机场(机场场景)*
void	**startReturnToDockAlternateLandPoint**(CommonCallbacks.CompletionCallback callback) *开始返航到机场备降点(机场场景)*
void	**setMaxFlightHeight** (int maxHeight CommonCallbacks.CompletionCallback callback) 设置最大飞行高度 maxHeight:范围5-500,单位 米（m）

void	**getMaxFlightHeight**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 返回设置的最大飞行高度，单位 米（m）
void	**setMaxFlightRadius**(int maxRadius, CommonCallbacks.CompletionCallback callback) 设置的最远飞行距离 maxRadius：范围 15-8000，单位 米（m）
void	**getMaxFlightRadius**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 获取设置的最远飞行距离，单位 米（m）
void	**setMaxFlightRadiusLimitationEnabled** (boolean enabled, CommonCallbacks.CompletionCallback callback) 设置限距开关 enabled: true限距开，已设置的最远距离值有效；false限距关，距离不受最远距离限制
	setMaxFlightRadiusLimitationEnabled (boolean enabled, CommonCallbacks.CompletionCallback callback) 设置限距开关 enabled: true限距开，已设置的最远距离值有效；false限距关，距离不受最远距离限制
void	**getMaxFlightRadiusLimitationEnabled** (CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取限距开关
void	**setHomeLocation**(LocationCoordinate2D homeLocation, CommonCallbacks.CompletionCallback callback) 设置飞机返航点
void	**setDockAlternateLandPoint**(LocationCoordinate2D homeLocation, CommonCallbacks.CompletionCallback callback) 设置飞机备用返航点(机场场景)
void	**setGoHomeHeightInMeters** (short backHeight, CommonCallbacks.CompletionCallback callback) 设置相对于飞机起飞位置的最低返航高度, 如果飞机当前的高度高于最低返航高度，它将

	以当前的高度返航。 backHeight：高度范围为5m ~ 500m
void	**getGoHomeHeightInMeters**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 获取相对于飞机起飞位置的最低返航高度,高度范围为5m ~ 500m
void	**setReturnToDockAlternateHeight**(int height, CommonCallbacks.CompletionCallback callback) 设置飞往机场备降点的高度（height：单位 cm，机场场景）
void	**getReturnToDockAlternateHeight**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 获取飞往机场备降点的高度，单位 cm
void	**startPrecisionGoHome**(CommonCallbacks.CompletionCallback callback) 开始精准返航，确保返航点有视觉标定的二维码
void	**cancelPrecisionGoHome**(CommonCallbacks.CompletionCallback callback) 取消精准返航
void	**setPrecisionGoHomeStateCallback**(PrecisionGoHomeState.Callback callback) 设置精准返航状态
void	**setConnectionFailSafeBehavior**(ConnectionFailSafeBehavior behavior, CommonCallbacks.CompletionCallback callback) 设置遥控器和飞机断开连接后的动作 ConnectionFailSafeBehavior HOVER:悬停 GO_HOME：返航。另有重载方法 **setConnectionFailSafeBehavior**(ConnectionFailSafeBehavior behavior, short loseTime, CommonCallbacks.CompletionCallback callback)，设置失联触发时间 loseTime（单位秒，取值范围 5~30；累计失联时间由固件固定为 60 秒）
void	**getConnectionFailSafeState(**CommonCallbacks.CompletionCallbackWith<ConnectionFailSafeState> callback) 获取失联动作与失联触发时间 loseTime（单位秒，返回 ConnectionFailSafeState 含 behavior 与 loseTime）；

	getConnectionFailSafeBehavior(CommonCallbacks.CompletionCallbackWith<ConnectionFailSafeBehavior> callback) 获取遥控器和飞机断开连接后的动作
void	**setLowBatteryWarningThreshold**(int percent, CommonCallbacks.CompletionCallback callback) 设置低电量告警阈值 Percent:15-50
void	**getLowBatteryWarningThreshold**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 获取低电量告警阈值
void	**setSeriousLowBatteryWarningThreshold**(int percent, @Nullable CommonCallbacks.CompletionCallback callback) 设置严重低电量告警阈值 percent 10-45
void	**getSeriousLowBatteryWarningThreshold**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 获取严重低电量告警阈值
void	**startTapFly**(LocationCoordinate3D point, float horizonSpeed, float verticalSpeed, CommonCallbacks.CompletionCallback callback) 开始指点飞行 point：目标点经纬度和相对地面高度 horizonSpeed：水平速度 verticalSpeed：垂直速度
void	**stopTapFly**(CommonCallbacks.CompletionCallback callback) 停止指点飞行
void	**setTapFlyStateCallback**(TapFlyState.Callback callback) 设置指点飞行状态监听
void	**setStateCallback**(FlightControllerState.Callback callback) 设置飞控状态监听 FlightControllerState:飞控状态信息类
	setSmartReturnToHomeEnabled(boolean enable, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置低电量返航开关 enable: ture为低电量返航开 false 为低电量返航关
	getSmartReturnToHomeEnabled(CommonCallbacks.CompletionCallbackWith<Boolean>

	paramCompletionCallbackWith) 获取低电量返航开关
RTK	**getRTK**() 获取RTK定位对象 RTK:定位对象
Simulator	**getSimulator**() 获取模拟器对象 Simulator:模拟器对象
void	**setVirtualStickModeEnabled**(boolean paramBoolean, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置虚拟摇杆模式开关
void	**getVirtualStickModeEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> paramCompletionCallbackWith) 获取虚拟摇杆模式开关
boolean	**isVirtualStickControlModeAvailable**() 获取虚拟摇杆是否可用，需要满足以下条件才可使用 1.开启虚拟摇杆模式 2.没有执行Waypoint，FollowMe，Hotpoint任务 3.处于GPS模式
VerticalControlMode	**getVerticalControlMode**() 获取垂直方向控制模式VerticalControlMode VELOCITY：速度模式，正垂直速度和负垂直速度分别用于飞机上升和下降。最大垂直速度定义为5米/秒。最小垂直速度定义为-3米/秒。 POSITION：位置模式（暂不支持）
void	**setVerticalControlMode**(VerticalControlMode paramVerticalControlMode) 设置垂直方向控制模式
RollPitchControlMode	**getRollPitchControlMode**() 获取水平方向控制模式RollPitchControlMode VELOCITY：速度模式，在机体坐标系中，正俯仰速度和负俯仰速度分别为飞机沿俯仰轴和y轴向正方向或负方向运动。正横滚转速度和负横滚转速度分别是飞机沿横滚转轴和x轴向正方向或负方向运动时的速度。最大速度定义为10米/秒。最小速度定义为-10米/秒。 ANGLE：角度模式（暂不支持）
void	**setRollPitchControlMode**(RollPitchControlMode paramRollPitchControlMode) 设置水平方向控制模式

YawControlMode	**getYawControlMode**() 获取偏航角模式 YawControlMode ANGULAR_VELOCITY: 角速度，正负角速度分别为飞机顺时针和逆时针旋转时的角速度。最大偏航角速度定义为100度/秒。最小偏航角速度定义为-100度/秒。 ANGLE: 角度模式（暂不支持）
void	**setYawControlMode**(YawControlMode paramYawControlMode) 设置偏航角模式
FlightCoordinateSystem	**setRollPitchCoordinateSystem**(FlightCoordinateSystem paramFlightCoordinateSystem) 设置水平方向坐标模式FlightCoordinateSystem GROUND: 地面坐标系（暂不支持） BODY: 机体坐标系
void	**getRollPitchCoordinateSystem**() 获取水平方向坐标模式
void	**sendVirtualStickFlightControlData**(FlightControlData paramFlightControlData, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置虚拟摇杆数据 FlightControlData： pitch：飞机沿着x轴（机头）速度(m/s)或俯仰的角度值(度)。 roll: 飞机沿着y轴（机右）速度(m/s)或角度值(度)滚转。 yaw: 飞机的偏航设置角速度(度/秒)或角度(度)值。 verticalThrottle: 飞机的垂直控制设置飞机的速度(m/s)或高度(m)值。
void	**changeYawAngular**(RotationMode mode, int angle, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置飞机机头航线角 RotationMode.ABSOLUTE_ANGLE 绝对角度模式 RotationMode.RELATIVE_ANGLE 相对角度模式 angle 角度 [-180 180]
void	**setHorizontalSpeed**(short xSpeed, short ySpeed) 设置飞机水平速度飞行 X轴为机头-机尾轴且机头方向为正， Y轴为机身左-右轴，且右方向为正 xSpeed X轴速度 cm [-2000,2000] ySpeed Y轴速度 cm [-2000,2000]
void	**stopHorizontalSpeed**() 取消飞机水平速度设置
void	**setVerticalSpeed**(short speed) 设置飞机垂直飞行速度 cm [-500,500] 向上为正 注意请勿使用此控

	制降落到地面 无法触发缓降 可能损坏飞机 降落使用一键降落
void	**stopVerticalSpeed**() 取消飞机水平速度设置
void	**confirmSmartReturnToHomeRequest**(boolean isConfirm, @androidx.annotation.Nullable CommonCallbacks.CompletionCallback callback) *是否确认智能返航请求*
void	**setBackHomeAction**(byte index, CommonCallbacks.CompletionCallbackWith<Byte> callback) *设置返航行为Index: 0.返航降落（默认） 1.返航不降落*
void	**getBackHomeAction**(CommonCallbacks.CompletionCallbackWith<Integer> callback) *获取返航行为Index: 0.返航降落（默认） 1.返航不降落*
void	**setFlyModeState**(boolean isOpen, CommonCallbacks.CompletionCallbackWith<Boolean> callback) *允许切换飞行模式开关*
void	**getFlyModeState**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) *获取当前飞行模式开关*
void	**setTripodMode**(boolean isOpen, CommonCallbacks.CompletionCallbackWith<Boolean> callback) *设置是否开启三脚架模式*
void	**getTripodMode**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) *获取三脚架模式*
Compass	**getCompass**() *获取磁力计*
FlightAssistant	**getFlightAssistant**() *智能飞行辅助类*
void	**getDroneBackInfo**(CommonCallbacks.CompletionCallbackWith<DroneBackInfo> callback) *获取返航信息*
void	**setBackHomeSpeed**(int backSpeed, CommonCallbacks.CompletionCallbackWith<Boolean> callback) *设置返航速度backSpeed 单位 m/s*

void	**checkIMUCalibration**(byte status, CommonCallbacks.CompletionCallbackWith<Byte> callback) *IMU校准:0 停止 1 开始*
void	**addIMUCalibrationCallback**(CommonCallbacks.CompletionCallbackWith<Integer> callback) *IMU 校准监听（已废弃，建议使用 setIMUStateCallback，详见 2.4.1.10 IMUState）*
void	**setAircraftOperationScenarioMode**(@NonNull AircraftOperationScenarioInfo mode, CommonCallbacks.CompletionCallback callback) *设置飞机的使用场景*
void	**setReturnRemainPower**(@IntRange(from = 0L, to = 100L) int remainPower,@Nullable CommonCallbacks.CompletionCallback callback) *设置返航预留电量*
void	**getReturnRemainPower**(@NonNull CommonCallbacks.CompletionCallbackWith<Integer> callback) *获取返航预留电量*

2.4.1.1 FlightControllerState

FlightControllerState表示飞行控制器的当前状态

限定符和类型	方法和说明
Attitude	**getAttitude**() 获取飞行器的姿态信息 pitch：俯仰角度（-180~180） roll：横滚角度（-180~180） yaw: 航向角（-180~180）
GoHomeExecutionState	**getGoHomeExecutionState**() 获取飞行器的返航状态 NOT_EXECUTING: 未执行返航 GO_UP_TO_HEIGHT：上升到返航高度 TURN_DIRECTION_TO_HOME_POINT：转向Home点 AUTO_FLY_TO_HOME_POINT：飞向返航点 GO_DOWN_TO_GROUND：Home点降落
LocationCoordinate2D	**getHomeLocation**() 获取返航点位置 latitude：纬度 longitude: 经度
LocationCoordinate3D	**getAircraftLocation**() 获取飞行当前位置 latitude：纬度 longitude: 经度 altitude：相对起飞点高度，单位（cm）
FlightMode	

	getFlightMode() 获取飞行模式 ATTI：姿态模式 GPS_NORMAL：标准模式 GPS_SPORT：运动模式 VISUAL：视觉模式 TRIPOD：三角架模式
Boolean	**isHasReachedMaxFlightHeight**() 是否到达限定高度
Boolean	**isHasReachedMaxFlightRadius** 是否到达限定距离
Boolean	**isHomeLocationSet** home点是否已设定
Boolean	**isGoingHome** 是否在返航中
Boolean	**isFlying** 是否在飞行中
Boolean	**isMotorsOn** 电机是否解锁
Int	**getGoHomeHeight** 获取返航高度，单位（cm）
Int	**getSatelliteCount** 获取当前飞行搜星颗数
float	**getVelocityX** 当前飞机在x方向上的速度，单位是厘米/秒，使用N-E-D(北东地坐标系)坐标系统
float	**getVelocityY** 当前飞机在y方向上的速度，单位是厘米/秒，使用N-E-D(北东地坐标系)坐标系统。
float	**getVelocityZ** 飞机在z方向上的当前速度，单位是厘米/秒，使用N-E-D(北东地坐标系)坐标系统。
int	**getFlightTimeInSeconds** 获取当次飞行时间，单位 秒（s）
FlightState	**getFlightState** 获取飞行状态 FLING：飞行中 GROUND：地面上 TAKEOFF：起飞中 LANDING：降落中 HOVERING：悬停中 BACKING：返航中
Boolean	**isSimulatorActive** 模拟飞行是否开启
float	**getDistance** 获取飞行距离，单位（cm）
float	**getTakeoffLocationAltitude** 获取起飞点的椭球高
GoHomeAssessment	**getGoHomeAssessment** 获取智能返航的数据 timeNeededToLandFromCurrentHeight：降落到地面需要时间 batteryPercentageNeededToGoHome：返航电池电池电量百分比 batteryPercentageNeededToLandFromCurrentHeight：降落到地面所需电量百分比 smartRTHCountdown：智能返航倒计时
float	**getUltrasonicHeightInMeters**() 获取超声高度，单位（cm）

2.4.1.2 RTK

RTK实时定位

限定符和类型	方法和说明
void	**setReferenceStationSource**(ReferenceStationSource currentReferenceSource, CommonCallbacks.CompletionCallback callback) 设置RTK通信类型 BASE_STATION: 基站RTK CUSTOM_NETWORK_SERVICE: 自定义网络RTK ONBOARD_RTK: 机载RTK
void	**setStateCallback**(RTKState.Callback callback) 设置RTK状态监听 RTKState：rtk状态信息
void	connectRtk(ReferenceStationSource currentReferenceSource, NetworkServiceSettings settings,GduRtkManager.OnRtkConnectListener listener) *连接RTKcurrentReferenceSource 当前参考站源settings 网络服务设置（当currentReferenceSource为CUSTOM_NETWORK_SERVICE时需要设置）*
void	disconnectRtk() *断开RTK连接*

2.4.1.3 RTKState

RTKState状态信息

限定符和类型	方法和说明
PositioningSolution	**getPositioningSolution** 获取RTK定位状态 NONE：无定位状态 SINGLE_POINT：单点定位 FLOAT：浮点定位 FIXED_POINT：固定解定位
HeadingSolution	**getHeadingSolution** 获取RTK定向状态 NONE：无定位状态 SINGLE_POINT：单点定位 FLOAT：浮点定位 FIXED_POINT：固定解定向
ReceiverInfo	**getMsReceiver1GPSInfo** 主天线GPS信息 satelliteCount:GPS星数
ReceiverInfo	**getMsReceiver1BeiDouInfo** 主天线北斗信息 satelliteCount:北斗星数
ReceiverInfo	**getMsReceiver1GLONASSInfo** 主天线GLONASS信息 satelliteCount: GLONASS星数
ReceiverInfo	**getMsReceiver1GalileoInfo** 主天线Galileo信息 satelliteCount: Galileo星数

ReceiverInfo	**getMsReceiver2GPSInfo** 从天线GPS信息 satelliteCount:GPS星数
ReceiverInfo	**getMsReceiver2BeiDouInfo** 从天线北斗信息 satelliteCount:北斗星数
ReceiverInfo	**getMsReceiver2GLONASSInfo** 从天线GLONASS信息 satelliteCount: GLONASS星数
ReceiverInfo	**getMsReceiver2GalileoInfo** 从天线Galileo信息 satelliteCount: Galileo星数
ReceiverInfo	**getBsReceiverGPSInfo** 基站GPS信息 satelliteCount:GPS星数
ReceiverInfo	**getBsReceiverBeiDouInfo** 基站北斗信息 satelliteCount:北斗星数
ReceiverInfo	**getBsReceiverGLONASSInfo** 基站GLONASS信息 satelliteCount: GLONASS星数
ReceiverInfo	**getBsReceiverGalileoInfo** 基站Galileo信息 satelliteCount: Galileo星数
LocationCoordinate2D	**getMsFusionLocation** 获取RTK计算的融合经纬度 Latitude：融合纬度 Longitude：融合经度
float	**getMsFusionAltitude** 获取RTK融合椭球高，单位（cm）
LocationCoordinate2D	**getBsLocation** 获取基站的经纬度 Latitude：基站纬度 Longitude：基站经度
float	**getBsAltitude** 获取基站的椭球高，单位（cm）
float	**getTakeOffAltitude** 获取起飞点的椭球高，单位（cm）
float	**getEllipsoidHeight** 获取飞行器椭球高度，单位（cm）
float	**getAircraftAltitude** 获取飞行器海拔高度，单位（cm）

2.4.1.4 Simulator

Simulator飞行模拟器

限定符和类型	方法和说明
void	**start**(InitializationData initializationData, CommonCallbacks.CompletionCallback callback) 开启模拟飞行 InitializationData：模拟初始化数据
boolean	**isSimulatorActive**() 模拟飞行是否开启

2.4.1.5 InitializationData

InitializationData模拟飞行初始化数据

限定符和类型	方法和说明
LocationCoordinate3D	**getLocation** 模拟飞行位置信息 latitude: 模拟飞行纬度 longitude：模拟飞行器经度 altitude：模拟飞行器相对高度
short	**getHeadAngle** 起始航向角
byte	**getSatelliteCount** 模拟搜星数
PositioningSolution	**getLocationStatus** 定位状态 NONE：无定位状态 SINGLE_POINT：单点定位 FLOAT：浮点定位 FIXED_POINT：固定解定位

2.4.1.6 NetworkServiceSettings

NetworkServiceSettings：提供RTK服务器的网络服务的设置。

限定符和类型	方法和说明
void	**setServerAddress**(String serverAddress) 设置服务器IP地址或域名
void	**setPort**(int port) 设置服务器端口
void	**setUserName**(String userName) 设置用户名
void	**setPassword**(String password) 设置用户密码
void	**setMountPoint**(String mountPoint) 设置挂载点类型

2.4.1.7 FlightAssistant

智能飞行辅助功能，提供智能飞行的方法，避障，降落保护，返航避障等功能

限定符和类型	方法和说明
void	**setVisionSensingEnabled**(boolean enabled, CommonCallbacks.CompletionCallback callback) 设置视觉感知开关，视觉感知开启后才能开启避障功能
void	

	getVisionSensingEnabled(CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取视觉感知开关
void	**setObstacleAvoidanceStrategyEnabled**(boolean enabled, CommonCallbacks.CompletionCallback callback) 设置避障策略开关，策略开启后飞机将执行避障功能
void	**getObstacleAvoidanceStrategyEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取避障策略开关
void	**setUpwardVisionObstacleAvoidanceEnabled**(Boolean enabled, @Nullable CommonCallbacks.CompletionCallback callback) 设置上视避障开关
void	**getUpwardVisionObstacleAvoidanceEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取上视避障开关
void	**setHorizontalVisionObstacleAvoidanceEnabled**(boolean enabled, CommonCallbacks.CompletionCallback callback) 设置水平避障开关
void	**getHorizontalVisionObstacleAvoidanceEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取水平避障开关
void	**setVisualObstaclesAvoidanceDistance**(float distance, @NonNull GDUFlightAssistantObstacleSensingDirection direction, @Nullable CommonCallbacks.CompletionCallback callback) 设置各个方向避障的距离 distance:避障距离 单位m GDUFlightAssistantObstacleSensingDirection：避障方位 **Upward：**上视 **Downward：**下视 **Horizontal：**水平
void	**getVisualObstaclesAvoidanceDistance**(GDUFlightAssistantObstacleSensingDirection direction,

	CommonCallbacks.CompletionCallbackWith<Float> callback) 获取各个方向避障的距离
void	**setDownwardFillLightMode**(FillLightMode mode, CommonCallbacks.CompletionCallback callback) 设置下视补光灯的状态 FillLightMode **ON:** 开启补光灯 **OFF:** 关闭补光灯
void	**getDownwardFillLightMode**(CommonCallbacks.CompletionCallbackWith<FillLightMode> callback) 设置下视补光灯的状态
void	**setRTHObstacleAvoidanceEnabled**(boolean isOpen, CommonCallbacks.CompletionCallback callback) 设置返航避障开关
void	**getRTHObstacleAvoidanceEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取返航避障开关
void	**setLandingProtectionEnabled**(boolean isOpen, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置降落保护开关
void	**getLandingProtectionEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> paramCompletionCallbackWith) 获取降落保护开关

2.4.1.8 AircraftOperationScenarioInfo

AircraftOperationScenarioInfo飞机使用场景信息，用于设置飞机在机巢场景下的运行参数，包括机巢模式、机巢航向、备降点、起飞点坐标等。

限定符和类型	方法和说明
AircraftOperationScenarioMode	**getAircraftMode**() / **setAircraftMode**(AircraftOperationScenarioMode aircraftMode)*：机巢模式（必须设置）* 0：普通模式 1：机巢模式
boolean	**isDockHeadingValid**() / **setDockHeadingValid**(boolean dockHeadingValid)*：机巢航向有效标志，机库默认无效* 0：无效

	1：有效 飞机降落时请求设置给机库，下次飞机上线机库再设置回给飞机
double	**getHangarHeadingAngleRad**() / **setHangarHeadingAngleRad**(double hangarHeadingAngleRad)：*机巢航向角（单位 rad，缩放 1e2）* 取值范围：(-PI, PI] * 100 机库默认无效；飞机降落时请求设置给机库，下次飞机上线机库再设置回给飞机
boolean	**isAlternateLandingPointValid**() / **setAlternateLandingPointValid**(boolean alternateLandingPointValid)：*备降点有效标志（必须设置且有效）* 0：无效 1：有效
double	**getAlternateLandingPointLongitude**() / **setAlternateLandingPointLongitude**(double alternateLandingPointLongitude)：*备降点经度（单位 度，缩放 1e7）*
double	**getAlternateLandingPointLatitude**() / **setAlternateLandingPointLatitude**(double alternateLandingPointLatitude)：*备降点纬度（单位 度，缩放 1e7）*
int	**getStatus**() / **setStatus**(int status)：*状态位原始值*
boolean	**isTakeoffPointCoordinateValid**() / **setTakeoffPointCoordinateValid**(boolean takeoffPointCoordinateValid)：*起飞点坐标有效标志* 0：无效 1：有效
boolean	**isQuickTakeoffValid**() / **setQuickTakeoffValid**(boolean quickTakeoffValid)：*快速起飞有效标志* 0：无效 1：有效
DockType	**getDockType**() / **setDockType**(DockType dockType)：*机库类型* 1=K01, 2=K02, 3=K03, 5=K05, 21=K02P, 31=K03P, 51=K05P
double	**getTakeoffPointLongitude**() / **setTakeoffPointLongitude**(double takeoffPointLongitude)：*起飞点经度（单位 度，缩放 1e7）* 快速起飞有效时，起始点坐标也需要传
double	**getTakeoffPointLatitude**() / **setTakeoffPointLatitude**(double takeoffPointLatitude)：*起飞点纬度（单位 度，缩放 1e7）* 快速起飞有效时，起始点坐标也需要传
double	

	getTakeoffPointEllipsoidHeight() / **setTakeoffPointEllipsoidHeight**(double takeoffPointEllipsoidHeight)：*起飞点椭球高（单位 米，缩放 1e2）* 快速起飞有效时，起始点坐标也需要传

> ❗ **关键字段依赖说明：**
>
> - **备降点**：经纬度必须设置且 alternateLandingPointValid = true
> - **快速起飞**：quickTakeoffValid = true 时，起飞点坐标（经度/纬度/椭球高）也必须传
> - **机巢航向**：飞机降落时请求设置给机库，下次飞机上线机库再设置回给飞机
> - **机巢模式**：aircraftMode 必须设置

2.4.1.9 DockType 机库类型

限定符和类型	方法和说明
K01	K01，ID为1
K02	K02，ID为2
K03	K03，ID为3

2.4.1.10 IMUState

IMUState IMU校准状态信息（对齐 DJI MSDK dji.common.flightcontroller.imu.IMUState）。通过 **setIMUStateCallback**(IMUState.Callback callback) 设置监听，回调 onUpdate 返回 IMUState，包含校准状态、校准姿态和校准进度。

限定符和类型	方法和说明
CalibrationState	**getCalibrationState**() 获取IMU校准状态枚举 NONE(0)：未校准 CALIBRATING(1)：校准中 SUCCESSFUL(2)：校准成功 FAILED(3)：校准失败 UNKNOWN(255)：未知
CalibrationOrientation	**getCalibrationOrientation**() 获取IMU校准姿态枚举（下一步操作方向，某面朝下） NOSE_DOWN：机头朝下 TAIL_DOWN：机尾朝下 RIGHT_DOWN：右侧朝下 LEFT_DOWN：左侧朝下 BOTTOM_DOWN：底部朝下 TOP_DOWN：顶部朝下 UNKNOWN：未知
int	**getCalibrationProgress**() 获取IMU校准进度，取值范围 [1,100]；未在校准状态时为 -1（当前协议帧无进度字节，恒为 -1）

协议引导值说明：IMU 校准周期帧 frameContent[0] 中，255~250 依次表示六面校准的下一步操作方向（255：正面朝上，对应 BOTTOM_DOWN；254：右侧朝上，对应 LEFT_DOWN；253：机头朝上，对应 TAIL_DOWN；252：左侧朝上，对应 RIGHT_DOWN；251：机尾朝上，对应 NOSE_DOWN；250：底部朝上，对应 TOP_DOWN），引导阶段校准状态为 CALIBRATING；其余状态值 13/15/16 为校准失败，14 为校准成功，21 为主动停止。

2.4.1.11 ConnectionFailSafeBehavior

遥控器与飞行器失联后的保护动作枚举，对应 setConnectionFailSafeBehavior / getConnectionFailSafeBehavior。

枚举值	说明
HOVER	失联后悬停
GO_HOME	失联后返航（默认）

2.4.1.12 ConnectionFailSafeState

失联保护配置信息，通过 getConnectionFailSafeState(CommonCallbacks.CompletionCallbackWith<ConnectionFailSafeState> callback) 回调返回。

方法	说明
ConnectionFailSafeBehavior getBehavior()	失联动作：HOVER 悬停 / GO_HOME 返航
short getLoseTime()	单次失联触发时间（单位秒，取值范围 5~30；累计失联时间由固件固定为 60 秒）

2.4.2 GDURemoteController

GDURemoteController表示飞机的遥控器。提供更改物理遥控器设置的方法。

限定符和类型	方法和说明
String	**getRCSN**() 获取遥控器SN号
void	**setComponentListener**(ComponentListener componentListener) 设置组件监听器
void	**setAircraftMappingStyle**(AircraftMappingStyle style, CommonCallbacks.CompletionCallback callback) 设置自定义样式的映射 AircraftMappingStyle: STYLE_1: 日本手 STYLE_2: 美国手 STYLE_3: 中国手

void	**getAircraftMappingStyle**(CommonCallbacks.CompletionCallbackWith<AircraftMappingStyle> callback) 获取自定义样式映射
void	**startPairing**(CommonCallbacks.CompletionCallback callback) 发送遥控器和飞行对频指令
void	**setChargeRemainingCallback**(BatteryState.Callback callback) 设置遥控器电量监听 BatteryState：电池状态 remainingChargeInPercent：剩余电量百分比
void	**setMultiControlMode**(MultiControlMode controlMode, CommonCallbacks.CompletionCallback callback) 设置多控模式
void	**getMultiControlMode**(CommonCallbacks.CompletionCallbackWith<MultiControlMode> callback) 获取当前多控模式
void	**getDroneList**(CommonCallbacks.CompletionCallbackWith<List<MultiControlInfo>> callback) 获取多控目标中飞机列表
void	**getRCList**(CommonCallbacks.CompletionCallbackWith<List<MultiControlInfo>> callback) 获取多控目标中遥控器列表
void	**setDroneControlEnable**(int droneId, CommonCallbacks.CompletionCallback callback) 设置多控某个飞机的控制权
void	**getDroneControlEnable**(int droneId, CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取多控某个飞机的控制权
void	**setDroneLivingEnable**(int droneId, CommonCallbacks.CompletionCallback callback) 设置多控指定飞机的视频流是否可用
void	**getDroneLivingEnable**(int droneId, CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取多控指定飞机的视频流是否可用
void	**startRCMedianCalibration**(CommonCallbacks.CompletionCallback callback) 开始遥控器中位校准
void	**startRCMaximumCalibration**(CommonCallbacks.CompletionCallback callback) 开始遥控器最大校准
void	**stopRCCalibration**(CommonCallbacks.CompletionCallback callback) 停止遥控器校准
void	**setRCControlRodValue**(short rollV, short pitchV, short acceleratorV, short headingV, CommonCallbacks.CompletionCallback callback) 设置遥控器RC控制杆量
void	**setContinueSendRCControlMidValueEnable**(boolean isEnable, CommonCallbacks.CompletionCallback callback) 设置是否持续发送遥控器中值(无摇

	杆的遥控器使用)
void	**getContinueSendRCControlMidValueEnable**(CommonCallbacks.CompletionCallbackWith<Boolean> callback) 获取当前是否在发送遥控器中值(无摇杆的遥控器使用)
void	**setCalibrationStateCallback**(CalibrationState.Callback callback) *设置遥控器校准监听*
void	**setRCStateCallback**(RCState.Callback callback) 设置遥控器状态监听，回调返回移动站使能、经纬度和朝向
void	**getDRTK**() 获取基站DRTK管理对象，用于设置DRTK模式、基站坐标与自校准
void	**connectDRTK**(CommonCallbacks.CompletionCallback callback) 请求地端NPS连接（图传盒子场景）

2.4.2.1 DRTK

DRTK为地面端差分定位管理对象，可进行状态监听、模式设置、基站坐标设置以及机库自校准。

限定符和类型	方法和说明
void	**setDRTKStateCallback**(DRTKStateInfo.Callback callback) 设置DRTK状态监听，回调返回UTC时间、经纬度、定位状态、卫星数和海拔
void	**setNestDRTKSelfCalibrateCallback**(DRTKSelfCalibrateCallback callback) 设置机库DRTK自标定状态监听
void	**startNestDRTKSelfCalibra**(String ipAddress, String port, String userID, String password, String mountedPoint, CommonCallbacks.CompletionCallback callback) 启动Nest DRTK自校准并配置NTRIP参数
void	**stopNestDRTKSelfCalibra**(CommonCallbacks.CompletionCallback callback) 停止Nest DRTK自校准
void	**setDRTKBaseStationCoord**(DRTKBaseStationCoord coord, CommonCallbacks.CompletionCallback callback) 设置DRTK基站坐标（纬度、经度、海拔）
void	**getDRTKBaseStationCoord**(CommonCallbacks.CompletionCallbackWith<DRTKBaseStationCoord>()) 获取DRTK基站坐标
void	**setDRTKMode**(DRTK.DRTKMode mode, CommonCallbacks.CompletionCallback callback) 设置DRTK模式：BASE_STATION（基准站）、MOBILE_STATION（移动站）
void	**getDRTKMode**(CommonCallbacks.CompletionCallbackWith<DRTK.DRTKMode> callback) 获取DRTK模式

2.4.2.2 DRTKSelfCalibrateCallback

DRTK自校准回调接口。

限定符和类型	方法和说明
void	**onCalibrateStart**() 自校准开始回调
void	**onCalibrateStatus**(DRTKSelfCalibrateCallback.Status status) 自校准状态回调
void	**onCalibrateEnd**() 自校准结束回调
Status	**DRTKSelfCalibrateCallback.Status**：**UNKNOWN** 未知/未准备好，**IN_PROGRESS** 校准进行中，**SUCCESS** 校准成功，**FAILED** 校准失败，**RETRY_REQUIRED** 需要重试

2.4.2.3 DRTKBaseStationCoord

DRTK基站坐标对象。

限定符和类型	方法和说明
double	**getLatitude**() 获取基站纬度
double	**getLongitude**() 获取基站经度
float	**getAltitude**() 获取基站海拔高度

2.4.3 GDUBattery

GDUBattery管理连接飞行器电池的信息和实时状态

限定符和类型	方法和说明
void	**setStateCallback**(BatteryState.Callback callback) 设置电池的信息和实时状态监听BatteryState电池实时状态信息
void	**getSerialNumber**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取电池的序列号
void	**getFirmwareVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取电池的版本号

2.4.3.1 BatteryState

BatteryState 电池实时状态信息

限定符和类型	方法和说明
int	**getFullChargeCapacity**() 获取电池总电量 mah
int	**getChargeRemaining**() 获取电池的剩余容量 mah
int	**getVoltage**() 获取电池当前电压MV
int	**getCurrent**() 获取电池当前电流MA
int	**getChargeRemainingInPercent**() 获取电量百分比 0-100
float	**getTemperature**() 获取电池温度
int	**getNumberOfDischarges**() 获取电池充放电循环次数
ConnectionState	**getConnectionState**() 获取连接状态 NORMAL: 正常 INVALID：无效 EXCEPTION：异常 UNKNOWN：未知
List<Integer>	**getCellVoltages**() 获取电芯电压列表

2.4.3 GDUAirLink

GDUAirLink图传链路，管理飞行器与遥控器，飞行器与移动设备之间无线链路通信

限定符和类型	方法和说明
void	**setUplinkSignalQualityCallback**(SignalQualityCallback signalQualityCallback) 设置链路信号质量监听 SignalQualityCallback.onUpdate(int uplinkQuality,int downlinkQuality) uplinkQuality：上行图传信号质量 downlinkQuality：下行图传信号质量
void	**isConnected**() 判断链路是否连接
void	**getFirmwareVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取图传的版本号
void	**getGroundImageTransmissionApplicationVersion**(CommonCallbacks.CompletionCallbackWith<String> callback)

	获取地面端图传应用程序版本号
void	**getGroundImageTransmissionSystemVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取*地面端图传系统版本号*
void	**getOnboardImageTransmissionSystemVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) *获取天空端图传系统版本号*
void	**getOnboardImageTransmissionApplicationVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) *获取天空端图传应用程序版本号*

2.4.3.1 LTEManager

LTEManager用于管理4G/5G备用链路（MQTT连接、服务地址配置、视频流地址、链路切换和状态回调）。可通过GDUAircraft.getLTE()或直接构造LTEManager(context)获取。

限定符和类型	方法和说明
void	**connectMqttService**(CommonCallbacks.CompletionCallback callback) 连接/初始化MQTT服务并回调结果
boolean	**isConnected**() 判断MQTT客户端是否已连接
void	**setAirlinkType**(AirlinkType airlinkType, CommonCallbacks.CompletionCallback callback) 设置图传链路模式（SDR/4G/自动等）
void	**setLTELinkInfoCallback**(LTELinkInfo.Callback callback) 设置LTE状态变化回调，回调LTELinkInfo对象
void	**setMqttServiceUrl**(String mqttServiceUrl, CommonCallbacks.CompletionCallback callback) 设置MQTT服务地址并下发到飞机
void	**getMqttServiceUrl**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取当前MQTT服务地址（从飞机端读取）

void	**setVideoServiceUrl**(String videoServiceUrl, CommonCallbacks.CompletionCallback callback) 设置视频流服务器地址
void	**getVideoServiceUrl**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取视频流服务器地址（返回带SN的地址）
void	**setLTEPushMode**(LTEPushMode mode, CommonCallbacks.CompletionCallback callback) 设置LTE推流模式
void	**getLTEPushMode**(CommonCallbacks.CompletionCallbackWith<LTEPushMode> callback) 获取LTE推流模式
void	**disconnect**() 断开MQTT连接并释放客户端资源

2.4.3.1.1 LTEManager 说明与行为要点

- connectMqttService 会使用AirlinkUtils提供的SN作为clientId并与配置的Broker建立连接，连接成功后会订阅飞机上行主题以接收来自飞机的LTE数据。
- push/接收的数据通过MQTT主题 gduraw/aircraft2server/datalinkcmd/ 和 gduraw/app2server/datalinkcmd/ 进行转发（需要设备端与服务端配合）。
- setAirlinkType 会调用链路切换接口(linkModeSetting)下发给飞机；在未连接或条件不满足时会返回对应错误码（如 GDUError.COMMON_DISCONNECTED 或 TIMEOUT）。

2.4.3.1.2 LTELinkInfo

LTELinkInfo 表示飞机端4G/5G链路状态汇总，通过回调返回。

限定符和类型	方法和说明
LTELinkNetworkType	**getNetworkType**() / **setNetworkType**(LTELinkNetworkType type) 网络类型（2G/3G/4G/5G）
LTELinkSignalQualityLevel	**getSignalQuality**() / **setSignalQuality**(LTELinkSignalQualityLevel level) 信号质量等级
LTELinkType	**getLinkType**() / **setLinkType**(LTELinkType type) 链路类型
LTEConnectStatus	

	getConnectStatus() / **setConnectStatus**(LTEConnectStatus status) MQTT连接状态
LTECurrentType	**getCurrentType**() / **setCurrentType**(LTECurrentType type) 当前启动类型（SDR/SDR_LTE等）
LTELinkPushStatus	**getNetPushType**() / **setNetPushType**(LTELinkPushStatus status) 网络推流状态
LTELinkAbnormalType	**getAbnormalType**() / **setAbnormalType**(LTELinkAbnormalType type) 模块异常类型
LTELinkPushStreamType	**getPushStreamType**() / **setPushStreamType**(LTELinkPushStreamType type) 推流方式（RTMP/WebRTC）

2.4.3.1.3 常用枚举说明

枚举	值	说明
LTELinkNetworkType	NO_SERVICE (0), NET_2G (1), NET_3G (2), NET_4G (3), NET_5G (4), UNKNOWN (-1)	LTE 网络类型
LTELinkSignalQualityLevel	POOR (1), MODERATE (2), GOOD (3), GREAT (4), UNKNOWN_SIGNAL (5), UNKNOWN_ERROR (10), UNKNOWN (-1)	信号质量等级（数值可对应大致 dBm：POOR <= -115，MODERATE <= -105，GOOD <= -95，GREAT > -85）
LTELinkType	SDR (0x01), FOUR_FIVE_G (0x02), AUTO_SDR_PRIORITY (0x03), AUTO_5G_PRIORITY (0x04), PURE_SDR (0x05), PURE_FOUR_FIVE_G (0x06), UNKNOWN (-1)	链路模式（SDR/4G/自动等）
LTEConnectStatus		MQTT 连接状态

	DISCONNECTED (0), CONNECTED (1), UNKNOWN (-1)	
LTECurrentType	IMAGE_TRANSMISSION (0x00), FOUR_G (0x01), UNKNOWN (-1)	当前启动图传类型
LTELinkPushStatus	NOT_STARTED (0), NORMAL (1), ABNORMAL (2), UNKNOWN (-1)	网络推流状态
LTELinkAbnormalType	NORMAL (0), MODULE_NOT_IDENTIFIED (1), SIM_CARD_NOT_DETECTED (2), BASE_STATION_REGISTER_ERROR (3), SERVER_CONNECT_FAILED (4), MQTT_COMMUNICATION_ERROR (5), SIM_CARD_LOOSE (6), SERVER_CONNECTION_LOST (7), UNKNOWN (-1)	模块/网络异常类型
LTELinkPushStreamType	RTMP (1), WEBRTC (2), UNKNOWN (-1)	推流方式（RTMP/WebRTC）
LTEPushMode	SAVE_DATA (0), NORMAL (1), UNKNOWN (-1)	LTE推流模式（省流模式/正常模式）

2.4.4 GDUGimbal

GDUGimbal提供了多种方法来控制云台，设置云台角度，云台校准等

限定符和类型	方法和说明
void	**setStateCallback**(GimbalState.Callback callback) 设置云台状态监听 GimbalState：云台状态信息
void	**startCalibration**(CommonCallbacks.CompletionCallback callback) 开始云台校准

void	**getFirmwareVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取图传的版本号
	getGimbalSN(CommonCallbacks.CompletionCallbackWith<String> callback) 获取云台SN
	reset(CommonCallbacks.CompletionCallback callback) 云台回中
	rotate(Rotation rotation, CommonCallbacks.CompletionCallback callback) 控制云台角度
GimbalType	getGimbalType() 获取云台类型 GimbalType:云台类型
DisplayMode	getSuppprtDisplayMode() 获取云台支持光类型 DisplayMode.THERMAL_ONLY 红外 DisplayMode.VISUAL_ONLY 可将光 DisplayMode.WAL 广角 DisplayMode.ZL 变焦 DisplayMode.PIP 分屏

2.4.4.1 GimbalState

GimbalState云台状态信息，包含云台的三轴角度

限定符和类型	方法和说明
Attitude	**getAttitudeInDegrees**() 云台姿态信息 Pitch:俯仰 Roll：横滚 Yaw：方位
void	**isCalibrating**() 云台是否校准中
void	**isCalibrationSuccessful**() 云台是否校准成功

2.4.5 GDUCamera

GDUCamera更改相机设置和执行相机操作的方法

限定符和类型	方法和说明
Attitude	**startShootPhoto**(CommonCallbacks.CompletionCallback callback) 在相机拍照模式下，相机开始拍照

void	**stopShootPhoto**(CommonCallbacks.CompletionCallback callback) 停止拍照
void	**startRecordVideo**(CommonCallbacks.CompletionCallback callback) 在录像模式下，相机开始录像
void	**stopRecordVideo**(CommonCallbacks.CompletionCallback callback) 停止当前录像
void	**setMode**(CameraMode cameraMode, CommonCallbacks.CompletionCallback completionCallback) 设置拍照或录像模式 CameraMode:拍照/录像模式 SHOOT_PHOTO：拍照模式 RECORD_VIDEO：录像模式
void	**getMode**(CommonCallbacks.CompletionCallbackWith<CameraMode> callback) 获取当前拍照/录像模式
void	**setStorageStateCallBack**(StorageState.Callback callback) 设置相机存储状态 StorageState：相机存储状态
void	**setSystemStateCallback**(SystemState.Callback callback) 设置相机状态信息监听 SystemState：相机状态信息
void	**getFirmwareVersion**(CommonCallbacks.CompletionCallbackWith<String> callback) 获取版本号
void	**formatSDCard**(CommonCallbacks.CompletionCallback callback) 格式化SD卡
void	**setHDLiveViewEnabled**(boolean enable, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置图传画面的视频流是否为高清画质 enable: true实时视图分辨率为1080p，帧率为30fps enable: false实时视图分辨率为720p，帧率为30fps
void	**getHDLiveViewEnabled**(CommonCallbacks.CompletionCallbackWith<Boolean> paramCompletionCallbackWith) 获取图传画面的视频流是否为高清画质
void	**isOpticalZoomSupported**() 是否支持光学变焦

void	**getOpticalZoomSpec** (CommonCallbacks.CompletionCallbackWith<SettingsDefinitions.OpticalZoomSpec> callback) 获取光学变焦参数
void	**setOpticalZoomFocalLength**(int focalLength, CommonCallbacks.CompletionCallback callback) 设置光学变焦焦距
void	**getOpticalZoomFocalLength**(CommonCallbacks.CompletionCallbackWith<Integer> callback) 获取光学变焦镜头焦距
void	**startContinuousOpticalZoom** (SettingsDefinitions.ZoomDirection direction, SettingsDefinitions.ZoomSpeed speed, CommonCallbacks.CompletionCallback callback) 开始连续光学变倍
void	**stopContinuousOpticalZoom**(CommonCallbacks.CompletionCallback paramCompletionCallback) 停止连续变倍
void	**setDisplayMode**(SettingsDefinitions.DisplayMode displayMode, CommonCallbacks.CompletionCallback callback) 设置相机的显示模式 DisplayMode: **THERMAL_ONLY**：单红外 **VISUAL_ONLY**：单可见光 **MSX**：融合 **WAL**：可见光定焦 **ZL**：可见光变焦 **PIP**：分屏 **OTHER**
void	**getDisplayMode**(CommonCallbacks.CompletionCallbackWith<SettingsDefinitions.DisplayMode> callback) 获取相机的显示模式
GDUMediaManager	**getMediaManager()** 获取相机媒体管理类
void	**setCaptureCameraStreamSettings**(@NonNull CameraStreamSettings streamSettings, @Nullable CommonCallbacks.CompletionCallback callback) *设置拍照时生效的多镜头存储配置*
void	**getCaptureCameraStreamSettings**(CommonCallbacks.CompletionCallbackWith<CameraStreamSettings> callback)

	获取拍照时生效的多镜头存储配置
void	**setRecordCameraStreamSettings**(@NonNull CameraStreamSettings streamSettings, @Nullable CommonCallbacks.CompletionCallback callback) *设置录像时生效的多镜头存储配置*
void	**getRecordCameraStreamSettings**(CommonCallbacks.CompletionCallbackWith<CameraStreamSettings> callback) *获取录像时生效的多镜头存储配置*
void	**setThermalPalette**(CameraThermalPalette palette, CommonCallbacks.CompletionCallback callback) 设置红外相机伪彩色（调色盘） 适用 GTIR800 红外云台，palette 取值见 2.4.5.6 CameraThermalPalette
void	**getThermalPalette**(CommonCallbacks.CompletionCallbackWith<CameraThermalPalette> callback) 获取红外相机当前伪彩色 适用 GTIR800 红外云台，回调返回 CameraThermalPalette 枚举
void	**setThermalMeasurementMode**(ThermalTemperatureMeasureMode mode, CommonCallbacks.CompletionCallback callback) 设置红外测温模式（测温分析类型） 适用 GTIR800 红外云台，mode 取值见 2.4.5.7 ThermalTemperatureMeasureMode：0关闭分析 1全屏测温 2区域测温1 3区域测温2 4区域测温3
void	**setThermalSpotMeteringTargetPoint**(short x, short y, CommonCallbacks.CompletionCallback callback) 设置红外测温光标点位置（点测温） 适用 GTIR800 红外云台，光标点X范围0-799，Y范围0-599（高字节在前） 需先开启测温总开关并设置测温模式
void	**setThermalMeteringArea**(short width, short height, short centerX, short centerY, CommonCallbacks.CompletionCallback callback) 设置红外测温矩形区域（区域测温） 适

	用 GTIR800 红外云台，宽度范围0-799，高度范围0-599，中心坐标X范围0-799，中心坐标Y范围0-599（高字节在前） 需将测温模式设置为区域测温（AREA_1/AREA_2/AREA_3）
void	**setThermalTemperatureDataEnabled**(boolean enabled, CommonCallbacks.CompletionCallback callback) 设置红外测温总开关 enabled: true开启（相机周期性上报温度信息） false关闭 开启后温度数据解析到 GlobalVariable.sInfraredCameraStatus
void	**getThermalTemperatureData**(CommonCallbacks.CompletionCallbackWith<TMSGimbalState> callback) 获取红外测温数据（最高温/最低温/中心温/光标温/区域平均温及各自坐标） 数据来源为相机周期性上报并缓存的 GlobalVariable.sInfraredCameraStatus，需先调用 setThermalTemperatureDataEnabled 开启测温 回调成功返回 TMSGimbalState，字段见 2.4.5.8 TMSGimbalState
boolean	**isFocalLengthListenerSupported()** 是否支持实时监听变焦镜头焦距
void	**setOpticalZoomFocalLengthCallback**(OpticalZoomFocalLengthCallback callback) 添加焦距变化监听器
void	**removeOpticalZoomFocalLengthCallback**(OpticalZoomFocalLengthCallback callback) 移除焦距变化监听器
void	**setZoom**(float ratio, CommonCallbacks.CompletionCallback callback) 设置变焦倍数，单位倍（x），取值范围见 getZoomRatioSpec()
float	**getOpticalZoomFactor()** 获取当前变焦倍数，单位倍（x）
boolean	**isZoomRatioListenerSupported()** 是否支持实时监听变焦倍数
void	**setMeteringMode**(MeteringMode mode, CommonCallbacks.CompletionCallback

	callback) 设置测光模式，取值见 MeteringMode 枚举
void	**getMeteringMode**(CommonCallbacks.CompletionCallbackWith<MeteringMode> callback) 获取当前测光模式
void	**setExposureMode**(ExposureMode mode, CommonCallbacks.CompletionCallback callback) 设置曝光模式，取值见 ExposureMode 枚举
void	**getExposureMode**(CommonCallbacks.CompletionCallbackWith<ExposureMode> callback) 获取当前曝光模式
void	**setVideoStreamType**(VideoStreamType type, CommonCallbacks.CompletionCallback callback) 设置视频码流类型（编码格式），H264 或 H265
void	**getVideoStreamType**(CommonCallbacks.CompletionCallbackWith<VideoStreamType> callback) 获取当前视频码流类型
void	**setFocusMode**(FocusMode mode, CommonCallbacks.CompletionCallback callback) 设置对焦模式，取值见 FocusMode 枚举（MANUAL/AUTO/AFC）
void	**setFocusTarget**(FocusTarget target, CommonCallbacks.CompletionCallback callback) 设置对焦目标点，坐标为归一化值 [0.0, 1.0]，(0.5, 0.5) 为画面中心
boolean	**isManualFocusSupported()** 是否支持手动对焦（可调节焦环值）

2.4.5.1 StorageState

StorageState相机存储状态

限定符和类型	方法和说明

CameraLight	**getLightType**() CameraLight存储图像模式 VISIBLE_LIGHT: 可见光 INFRARED_LIGHT：红外光
boolean	**isFormatted**() 相机SD卡是否格式化完成
boolean	**isFormatting**() 相机SD卡是否格式化中
boolean	**isFull**() 相机SD卡存储是否已满
boolean	**isInserted**() 相机是否插入SD卡
int	**getTotalSpace**() 获取总容量，单位M
int	**getRemainingSpace**() 获取剩余容量，单位M

2.4.5.2 SystemState

SystemState相机状态信息，包含拍照/录像模式，拍照状态，录像状态等

限定符和类型	方法和说明
CameraMode	**getMode**() CameraMode 相机模式 SHOOT_PHOTO: 拍照模式 RECORD_VIDEO：录像模式
boolean	**isRecording**() 相机是否在录像中
boolean	**isPhotoStored**() 相机照片存储完成
int	**getCurrentVideoRecordingTimeInSeconds**() 相机录像时间，单位秒（s）

2.4.5.3 GDUMediaManager

相机媒体管理 获取相机图片 视频文件

GDUCamera.getMediaMangager()

限定符和类型	方法和说明
void	**enable(CommonCallbacks.CompletionCallback callback)** 进入媒体下载模式
void	**disable(CommonCallbacks.CompletionCallback callback)** 退出媒体下载模式
void	**refreshMediaList(CommonCallbacks.CompletionCallback callback)** 刷新媒体库文件

void	**getMediaFileList(FileDownCallback.OnMediaListCallBack callBack)** 获取媒体库文件列表 **OnMediaListCallBack : onStart()开始获取: onGetMediaList(List<MediaFile> list)获取到的相册文件列表: onFail(GDUError error)获取出错MediaFile: 媒体文件数据path 文件原始完整路径name 文件名称 fileSize 原文件大小createTime 文件创建时间戳duration 视频文件长度 mediaType 文件类型 视频/图片**
void	**getThumbnail(String path,OnMediaImageCallback callback)获取缩略图 getPriview(String path ,OnMediaImageCallback callback)** 获取预览图 **path:** 图片路径 **OnMediaImageCallBack :onStart():onProgress(long total,long current)** 当前获取进度 **:onGetMediaImage(Bitmap bitmap, byte []bytes)** 图片bitmap **:onFail(GDUError error)** 获取异常信息

void	**getRawImage(String path, String saveDir, OnMediaFileCallback callback)** 获取图片原图 path: 图片路径 saveDir: 原图存储路径 原图较大 存成文件后返回文件路径 OnMediaImageCallBack : onProgress(long total,long current) 当前获取进度, total总长度，current当前下载长度 onProgress(int progress) 当前获取进度 onRealtimeDataUpdate(byte[]data ,long position)data 获取到的媒体文件数据 position 下载到的data在文件的偏移量 : onSuccess(String path)下载的原图存储路径: onFail(GDUError error) 获取异常信息
void	**getVideoFile(String path, OnMediaFileCallback callback)** 获取视频原文件
void	**getRawFileByWifi**(String path, String saveDir, FileDownCallback.OnMediaFileCallBack callBack) **path**: 图片路径 **saveDir**: 原图存储路径 原图较大 存成文件后返回文件路径 **OnMediaImageCallBack** : **onProgress**(long total,long current) 当前获取进度，total总长度，current当前下载长度 **onProgress**(int progress) 当前获取进度 **onRealtimeDataUpdate**(byte[]data ,long position)data 获取到的媒体文件数据 position 下载到的data在文件的偏移量 : **onSuccess**(String path)下载的原图存储路径: **onFail**(GDUError error) 获取异常信息 **注意：使用此方法时需引用 api 'commons-net:commons-net:3.10 库**
	public void getRawFileNoCheckByWifi(String path, String saveDir, FileDownCallback.OnMediaFileCallBack callBack) *通过WiFi下载原图和视频，不用Android来校验wifi的可用和联通性*

	path: 图片路径 **saveDir**: 原图存储路径 原图较大 存成文件后返回文件路径 **OnMediaImageCallBack** : **onProgress**(long total,long current) 当前获取进度，total总长度，current当前下载长度 **onProgress**(int progress) 当前获取进度 **onRealtimeDataUpdate**(byte[]data ,long position)data 获取到的媒体文件数据 position 下载到的data在文件的偏移量 : **onSuccess**(String path)下载的原图存储路径: **onFail**(GDUError error) 获取异常信息
void	**setVideoDataListener(VideoFeeder.VideoDataListener listener)** 设置视频文件播放监听 VideoDataListener: onReceive(byte[] datas, int len) datas: 播放视频的H264/H265数据 len: H264/H265数据长度
void	**playVideo (String path,CommonCallbacks.CompletionCallback callback)** 播放视频文件
void	**seekVideo(String path, int position,CommonCallbacks.CompletionCallback callback)** 指定视频播放位置 **position:** 指定播放时间s
void	**pauseVideo(String path,CommonCallbacks.CompletionCallback callback)** 暂停播放
void	**stopVideo(Sting path,CommonCallbacks.CompletionCallback callback)** 停止视频播放
void	deleteFile(String path, SocketCallBack3 socketCallBack3) 删除媒体文件
void	deleteFiles(List<MediaFile> files, final CommonCallbacks.CompletionCallbackWithTwoParam<List<MediaFile>, GDUError> callback) *删除指定的多个媒体文件。*
void	setVideoPlaybackListener(FileDownCallback.VideoPlaybackStateListener listener) *设置视频回放状态监听器。*

2.4.5.4 CameraStreamSettings

拍照或录像时的多镜头存储设置—— 可选择让哪些镜头进行拍照 / 录像并存储

通过 `Builder` 构建实例（构造方法为私有）。

限定符和类型	方法和说明
List<CameraVideoStreamSource>	**getCameraVideoStreamSources()** 获取需要拍照/录像的镜头源列表
boolean	**needCurrentLiveViewStream()** 是否保存当前图传实时画面 `true` ： 拍照/录像时额外存储当前

	liveview 流画面（分屏 PIP 下为截图）
Builder	**setNeedCurrentLiveViewStream**(boolean enable) 设置是否保存当前实时图传画面 `enable`： `true` 额外存储当前 liveview 画面
Builder	**setCameraVideoStreamSources**(List<CameraVideoStreamSource> sources) 设置需要拍摄的镜头源列表 `sources`： 参与拍照/录像的镜头源集合
CameraStreamSettings	**build()** 生成 CameraStreamSettings 实例

2.4.5.5 CameraVideoStreamSource

多光相机镜头流源（枚举）—— 标识参与拍照 / 录像 / 图传的具体镜头类型。

枚举值	说明
WIDE	可见光广角镜头
ZOOM	变焦镜头
INFRARED_THERMAL	红外热成像镜头
LASER_CAMERA	激光测距模块画面
UNKNOWN	未知镜头

2.4.5.6 CameraThermalPalette

红外相机伪彩色（调色盘）枚举，用于 GDUCamera.setThermalPalette / getThermalPalette。取值与 GDU 红外协议 IRSettingParam.pseudoColor 字段一致（0-9 共 10 种伪彩，默认值为 2 铁红）。

枚举值	值	说明
WHITE_HOT	0	白热，最常用，高温显示为白色
LAVA	1	熔岩
IRON_RED	2	铁红，默认值
HOT_IRON	3	热铁
MEDICAL	4	医疗
ICE_FIRE	5	北极

RAINBOW_1	6	彩虹1
RAINBOW_2	7	彩虹2
RED_HOT	8	描红
BLACK_HOT	9	黑热，高温显示为黑色
UNKNOWN	-1	未知

限定符和类型	方法和说明
int	**value()** 获取伪彩对应的协议值（0-9）
String	**description()** 获取伪彩描述（如"白热""铁红"）
CameraThermalPalette	**fromValue(int value)** 根据协议值获取对应的伪彩枚举；未匹配时返回 null

2.4.5.7 ThermalTemperatureMeasureMode

红外测温模式枚举，用于 GDUCamera.setThermalMeasurementMode。参照 DJI MSDK V4 ThermalMeasurementMode 设计，取值与 GDU 红外协议 setThermometryType 的 type 字段一致。

枚举值	值	说明
DISABLED	0	关闭分析
FULL_SCREEN	1	全屏测温，默认值
AREA_1	2	区域测温1
AREA_2	3	区域测温2
AREA_3	4	区域测温3
UNKNOWN	-1	未知

限定符和类型	方法和说明
int	**value()** 获取测温模式对应的协议值（0-4）
String	**description()** 获取测温模式描述（如"全屏测温""区域测温1"）

ThermalTemperatureMeasureMode	**fromValue(int value)** 根据协议值获取对应的测温模式枚举；未匹配时返回 null

2.4.5.8 TMSGimbalState

红外测温数据，GDUCamera.getThermalTemperatureData 回调返回。包含最高温、最低温、中心温、光标温、区域平均温及各自坐标（坐标 X 0-799，Y 0-599），数据由相机开启测温总开关后周期性上报并缓存到 GlobalVariable.sInfraredCameraStatus。

限定符和类型	方法和说明
float	**getHighestTemp()** 最高温度值（℃）
float	**getLowestTemp()** 最低温度值（℃）
float	**getCenterTemp()** 中心温度值（℃）
float	**getCursorPointTemp()** 光标点温度值（℃）
float	**getAreaAvgTemp()** 区域平均温度值（℃）
int	**getHighestTempPoint_X()** 最高温点X坐标（0-799）
int	**getHighestTempPoint_Y()** 最高温点Y坐标（0-599）
int	**getLowestTempPoint_X()** 最低温点X坐标（0-799）
int	**getLowestTempPoint_Y()** 最低温点Y坐标（0-599）
int	**getCenterTempPoint_X()** 中心温点X坐标（0-799）
int	**getCenterTempPoint_Y()** 中心温点Y坐标（0-599）
int	**getCursorPoint_X()** 光标测温点X坐标（0-799）
int	**getCursorPoint_Y()** 光标测温点Y坐标（0-599）
int	**getAreaAvg_X()** 区域测温点X坐标（0-799）
int	**getAreaAvg_Y()** 区域测温点Y坐标（0-599）
long	**getIrFreeCapacity()** 红外相机存储剩余容量
String	**getInfo()** 状态信息描述

2.4.5.9 MeteringMode 测光模式

枚举值	值	说明
CENTER_WEIGHTED	1	中央重点测光，侧重画面中央区域
SPOT	2	点测光，对指定小区域测光
AVERAGE	3	平均测光，全屏平均计算
UNKNOWN	0	未知

2.4.5.10 ExposureMode 曝光模式

枚举值	值	说明
PROGRAM	0	程序自动，快门和光圈均自动
MANUAL	1	手动曝光，快门、光圈、ISO 均手动控制
UNKNOWN	-1	未知

2.4.5.11 VideoStreamType 视频码流类型

枚举值	值	说明
H264	0	H.264 编码，兼容性好，默认值
H265	1	H.265 / HEVC 编码，压缩率更高
UNKNOWN	-1	未知

2.4.5.12 FocusMode 对焦模式

枚举值	值	说明
AUTO	0	自动对焦，可设置对焦目标点
MANUAL	1	手动对焦，通过焦环值调节焦距
UNKNOWN	-1	未知

2.5、MISSION CLASSES

2.5.1 MissionControl

MissionControl任务控制类用来获取任务执行控制类

限定符和类型	方法和说明
WaypointMissionOperator	**getWaypointMissionOperator**() WaypointMissionOperator 航点任务执行类
HotpointMissionOperator	getHotpointMissionOperator() *获取兴趣点环绕操作类*
FollowMeMissionOperator	getFollowMeMissionOperator() *获取GPS跟随操作类*

2.5.2 WaypointMissionOperator

WaypointMissionOperator航点任务执行类，用来控制，执行和监听航线航点任务。

限定符和类型	方法和说明
void	**addListener**(WaypointMissionOperatorListener operatorListener) WaypointMissionOperatorListener 航点任务执行监听
GDUError	**loadMission**(WaypointMission mission) 加载航迹任务 WaypointMission:航点任务
void	**uploadMission**(CommonCallbacks.CompletionCallback completionCallback) 上传航点任务到飞行器
void	**pushKMZFileToAircraft**(String missionFilePath, TaskType taskType，CommonCallbacks.CompletionCallbackWithProgress<Double> callback) *上传KMZ文件至飞机端* missionFilePath:航线的路径， TaskType：航线的类型
void	**startMission**(CommonCallbacks.CompletionCallback completionCallback) 开始航点任务
void	**startMissionWithTaskID**(long taskId, CommonCallbacks.CompletionCallbackWith<St

	ring> completionCallback) *开始指定任务id航迹* 返回任务名称：由时间戳和taskId组成，如：20260721162013_12345
void	**pauseMission**(CommonCallbacks.CompletionCallback completionCallback) 暂停航点任务
void	**resumeMission**(CommonCallbacks.CompletionCallback completionCallback) 继续航点任务
void	**stopMission**(CommonCallbacks.CompletionCallback completionCallback) 停止航点任务
WaypointMissionState	**getCurrentState**() 获取当前状态 READY_TO_UPLOAD UPLOADING READY_TO_EXECUTE EXECUTING EXECUTION_PAUSED UNKNOWN

2.5.2.1 TaskType

UploadPlanParam.TaskType 用于标识上传航线文件的任务类型，作为上传参数的一部分参与飞行器端的任务解析和执行。

枚举	值	说明
NORMAL_TASK	0	普通航迹航线（普宙1.0）
PRECISE_REPHOTO	1	精准复拍航线（普宙1.0）
DJI_TASK	2	大疆航线解析（kml,KMZ ）
GDU_V2_TASK	3	普宙2.0航线执行（wpml）
NORMAL_TASK_TAR_GZ	4	普通航迹航线（普宙1.0，xxx.tar.gz）

pushKMZFileToAircraft 上传 KMZ 文件时，通常使用 UploadPlanParam.TaskType.***DJI_TASK***，这是大疆航线场景所需的任务类型。

2.5.2.2 WaypointMission

WaypointMission航点任务，飞行器将在航点之间飞行，到达航点后执行动作，并在航点之间调整航向和高度。航点是飞机将飞行到的物理位置。创建一系列航点将为飞机规划一条飞行路线。航点可以添加动作，当飞机到达航点时执行。

限定符和类型	方法和说明
int	**setMissionID**() 设置任务ID
void	**setWaypointCount**() 设置航点总数
void	**setMaxFlightSpeed** 设置最大飞行速度，单位（m/s）
void	**setAutoFlightSpeed**() 设置自动飞行速度，单位（m/s）
void	**setFinishedAction**(WaypointMissionFinishedAction action) WaypointMissionFinishedAction：任务完成后动作 NO_ACTION: 悬停 GO_HOME：返航 AUTO_LAND：降落
void	**setHeadingMode**(WaypointMissionHeadingMode mode) WaypointMissionHeadingMode AUTO:自动 USING_INITIAL_DIRECTION：使用初始角度 USING_WAYPOINT_HEADING：使用航点角度
void	**setWaypointList**(List<Waypoint> list) 设置航点列表 Waypoint：航点信息
void	**setResponseLostActionOnRCSignalLost**() 设置失联是否响应失联行为
void	**setAltitudeMode**(int altitudeMode) *设置高程类型： 0相对高度 1椭球高 2海拔高度*
void	**setMissionExecutedWaypointCount**(int missionExecutedWaypointCount) 设置已执行的航点数，断点续飞时填入航点数，会从跳过这个航点之前的点位，直接从填入的航点数开始飞行

2.5.2.3 Waypoint

Waypoint表示航点任务中的目标点。航路点任务中一条飞行航线由多个航点组成。可以为每个Waypoint定义执行的操作。

限定符和类型	方法和说明
void	**setCoordinate**(LocationCoordinate2D coordinate) 设置航点的经纬度坐标 LocationCoordinate2D: latitude:纬度 longitude:经度
void	**setAltitude**(double altitude) 设置航点相对与起飞点的高度，单位 米(m)

void	**setHeading**(float heading) 设置飞机航向角度
void	**setUsingWaypointAutoFlightSpeed**(boolean usingWaypointAutoFlightSpeed) 设置是否使用自动飞行速度
void	**setUsingWaypointMaxFlightSpeed**(boolean usingWaypointMaxFlightSpeed) 设置是否使用最大飞行速度
void	**setAutoFlightSpeed**(float autoFlightSpeed) 设置自动飞行速度
void	**setMaxFlightSpeed**(float maxFlightSpeed) 设置最大飞行速度，单位（m/s）
void	**setGimbalPitch**(float gimbalPitch) 设置云台俯仰角度
void	**setWaypointActions**(List<WaypointAction> waypointActions) 设置航点子动作
void	**setIsSafeReturnPoint**(int type) 设置航点是否为安全点，在返航类型为安全点返航时，返航到安全点，飞机会拔高到返航高度，直线返航 SafePointReturn { *NOT_SAFE_POINT*(0), 非安全点 *SAFE_POINT*(1);} 安全点

2.5.2.4 WaypointAction

WaypointAction表示waypoint的一个航点操作。当飞机到达相应航点时执行什么动作。

限定符和类型	方法和说明
WaypointActionType	actionType STAY：悬停 START_TAKE_PHOTO：拍照 START_RECORD：录像 STOP_RECORD：停止录像 ROTATE_AIRCRAFT：旋转机头角度 GIMBAL_PITCH：旋转云台俯仰角度
int	actionParam actionType为STAY:悬停时间，单位s actionType为ROTATE_AIRCRAFT:旋转机头角度。范围（-180~180） actionType为GIMBAL_PITCH:旋转云台俯仰角。范围（-90~30）
String	

	fileSuffix 拍照自定义名称（可空）。对应协议 name 元素的 fileCode 属性，用作照片的名称，仅 START_TAKE_PHOTO（拍照）动作有效；传值时不要带扩展名（如 .jpg），否则生成的照片名会多出一截后缀，例如 名称.jpg_0001_1.JPG。照片按“自定义名称_序号_I.png”的普宙格式命名。

2.5.2.5 WaypointMissionOperatorListener

WaypointMissionOperatorListener航点任务监听，监听航点上传，执行和结束状态

限定符和类型	方法和说明
WaypointMissionOperator	**onUploadUpdate**(WaypointMissionUploadEvent event) 航点上传实时状态 WaypointMissionUploadEvent getProgress()：获取上传进度
void	**onExecutionUpdate**(WaypointMissionExecutionEvent event) 航点执行实时状态 WaypointMissionExecutionEvent航点执行状态
void	**onExecutionStart**() 航点开始执行反馈
void	**onExecutionFinish**(GDUError error) 航点结束反馈

2.5.2.6 WaypointMissionExecutionEvent

WaypointMissionExecutionEvent航点执行状态

限定符和类型	方法和说明
WaypointMissionState	**getCurrentState**() 航点执行状态 WaypointMissionState EXECUTING:航点执行中 EXECUTION_PAUSED:航点暂停
WaypointExecutionProgress	**getProgress**() 航点执行进度 WaypointExecutionProgress targetWaypointIndex:当前执行航点数 isWaypointReached: 是否到达航点 totalWaypointCount：总航点数

2.5.2.7 WaypointMissionFinishedAction

任务完成后动作

限定符和类型	方法和说明
NO_ACTION	**悬停**
GO_HOME	**直线返航**
AUTO_LAND	**自动降落**
RETURN_ALONG_ROUTE	**沿航线返航**：按航线反向返回起点/返航路线（返回路径沿原航线轨迹）
RETURN_TO_SAFE_POINT	**安全点返航**：返回到事先设置或计算的安全点（例如预设安全点或实时计算安全着陆点）

2.5.3 FollowMeMissionOperator

FollowMeMissionOperator用来控制，监听跟随任务的对象。通过MissionControl获取

限定符和类型	方法和说明
void	**addListener**(FollowMeMissionOperatorListener operatorListener) FollowMeMissionOperatorListener 跟随任务执行监听
void	**startMission**(CommonCallbacks.CompletionCallback completionCallback) 开始跟随任务
void	**stopMission**(CommonCallbacks.CompletionCallback completionCallback) 停止跟随任务
void	**updateFollowingTarget**(LocationCoordinate2D followingTarget, CommonCallbacks.CompletionCallback paramCompletionCallback) 更新GPS跟随点的坐标
void	**setFollowMeHeading**(FollowMeHeading heading, float angle, CommonCallbacks.CompletionCallback completionCallback) 设置当前GPS跟随的航向角度 **FollowMeHeading** GPS跟随的航向角类型 TOWARD_FOLLOW_POSITION：朝向目标点 CONTROLLED_BY_REMOTE_CONTROLLER：遥控器控航向角 SET_HEADING_ANGLE：设置航向

	角度 **angle**：航向角角度，当heading为SET_HEADING_ANGLE有效 范围（-180~180）
void	**setFollowMeGimbalPitch**(FollowMeGimbalPitch gimbalPitch, int pitch, CommonCallbacks.CompletionCallback completionCallback) **FollowMeGimbalPitch** GPS跟随云台俯仰角度 TOWARD_FOLLOW_POSITION：朝向目标点 SET_GIMBAL_PITCH：设置云台角度 CONTROLLED_BY_REMOTE_CONTROLLER: 打杆控航向 **pitch**：云台俯仰角度，当gimbalPitch为SET_GIMBAL_PITCH时有效, 范围（30~-90）

2.5.3.1 FollowMeMission

FollowMeMission跟随任务，开启跟随任务后，飞机将跟随并保持与GPS设备恒定距离。

限定符和类型	方法和说明
int	**FollowMeMission**(FollowMeHeading heading, double latitude, double longitude, float altitude) heading: 飞机跟随时的机头朝向 latitude: 飞机起飞点的纬度 longitude: 飞机起飞点的经度 altitude: 飞机起飞点的高度 isUseHighPrecision: 是否开启高精度跟随（地面端需安装RTK定位模块） isUseHighPrecision为true时需设置以下参数 isSetDistanceEnable: 是否设置相对距离 frontAndBackDistance: 前后相对距离（单位 m) leftAndRightDistance: 左右相对距离(单位 m) isSetHeightEnable: 是否设置相对高度 height: 相对高度 (单位 m)
void	**FollowMeHeading** 跟随时机头朝向 TOWARD_FOLLOW_POSITION：机头朝向目标 CONTROLLED_BY_REMOTE_CONTROLLER：遥控器控制机头朝向

2.5.3.2 FollowMeMissionOperatorListener

FollowMeMissionOperatorListener跟随任务状态监听，包括任务的开始，结束，执行状态反馈

限定符和类型	方法和说明

void	**onExecutionUpdate**(FollowMeMissionEvent event) 跟随任务执行实时状态 FollowMeMissionEvent跟随任务执行状态
void	**onExecutionStart**() 跟随任务开始执行反馈
void	**onExecutionFinish**(GDUError error) 跟随任务结束反馈

2.5.3.3 FollowMeMissionEvent

FollowMeMissionEvent跟随任务执行状态

限定符和类型	方法和说明
FollowMeMissionState	**getCurrentState**() 跟随任务执行状态 FollowMeMissionState EXECUTING:跟随任务执行中
float	**getDistanceToTarget**() 飞机与目标的距离

2.5.4 HotpointMissionOperator

HotpointMissionOperator用来控制，监听GPS环绕任务的对象。可通过MissionControl获取

限定符和类型	方法和说明
void	**addListener**(HotpointMissionOperatorListener operatorListener) HotpointMissionOperatorListener 环绕任务执行监听
void	**startMission**(HotpointMission hotpointMission, CommonCallbacks.CompletionCallback paramCompletionCallback) 开始环绕任务
void	**pause**(CommonCallbacks.CompletionCallback paramCompletionCallback) 暂停GPS环绕
void	**resume**(CommonCallbacks.CompletionCallback paramCompletionCallback) 继续GPS环绕
void	**stop**(CommonCallbacks.CompletionCallback completionCallback) 停止环绕任务

void	**setAngularVelocity**(float AngularVelocity, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置环绕角速度 rad/s, 1e2 取值范围[5,120]
void	**setRadius**(float radius, CommonCallbacks.CompletionCallback paramCompletionCallback) 设置环绕半径，单位 m，取值范围[5,500]
void	**resetHeading**(CommonCallbacks.CompletionCallback paramCompletionCallback) 重置机头角度
void	**setHotPointHeading**(HotpointHeading heading, float angle, CommonCallbacks.CompletionCallback completionCallback) 设置GPS环绕机头航向角度 **HotpointHeading**航向角度类型 ALONG_CIRCLE_LOOKING_FORWARDS：指向圆切线前进方向 ALONG_CIRCLE_LOOKING_BACKWARDS：背向圆切线前进方向 TOWARDS_HOT_POINT：指向圆心 AWAY_FROM_HOT_POINT：背向圆心 CONTROLLED_BY_REMOTE_CONTROLLER：遥控器控制 USING_INITIAL_HEADING：使用初始航向角度 SET_HEADING_ANGLE：设置指定角度的航向角 **angle:** 指定航向角角度 当heading为SET_HEADING_ANGLE时有效，范围（-180~180）
void	**setHotpointGimbalPitch**(HotpointGimbalPitch gimbalPitch, int pitch, CommonCallbacks.CompletionCallback completionCallback) 设置GPS环绕云台俯仰角度 **HotpointGimbalPitch** 云台俯仰角类型
	TOWARD_FOLLOW_POSITION：朝向目标的 SET_GIMBAL_PITCH：设置云台角度 CONTROLLED_BY_REMOTE_CONTROLLER：打杆控航向 **pitch:** 指定云台俯仰角度 当gimbalPitch为SET_GIMBAL_PITCH时有效，范围（30~-90）
HotpointMissionState	**getCurrentState**() 获取当前GPS环绕状态 CAN_NOT_START：当前状态禁止飞行 READY_TO_EXECUTE：飞机准备开始执行 EXECUTING：任务执行中

	EXECUTION_PAUSED：任务暂停执行 UNKNOWN：未知错误

2.5.3.1 HotpointMission

HotpointMission环绕任务，开启环绕任务后，飞机将围绕一个GPS指定点以恒定半径反复飞行。用户可以控制飞机以特定的半径和高度围绕GPS点飞行。在执行过程中，用户还可以使用控制器修改其半径和速度。

限定符和类型	方法和说明
int	**HotpointMission**(LocationCoordinate2D hotpoint, double altitude, double radius, float angularVelocity, boolean isClockwise, HotpointStartPoint startPoint, HotpointHeading heading) hotpoint: 环绕点的经纬度坐标 altitude: 飞机环绕高度 radius: 飞机环绕半径 angularVelocity: 飞机环绕角速度 isClockwise: 是否为顺时针 startPoint: 起飞点位置 heading: 机头朝向 **HotpointMission**(LocationCoordinate2D hotpoint, double altitude, double radius, float angularVelocity, boolean isClockwise,int num HotpointStartPoint startPoint, HotpointHeading heading) hotpoint: 环绕点的经纬度坐标 altitude: 飞机环绕高度 radius: 飞机环绕半径 angularVelocity: 飞机环绕角速度 isClockwise: 是否为顺时针 num:飞行圈数，0表示无限绕飞，大于0表示绕飞圈数 startPoint: 起飞点位置 heading: 机头朝向
void	**HotpointHeading**环绕时机头朝向 ALONG_CIRCLE_LOOKING_FORWARDS: 机头圆周前进的方向 ALONG_CIRCLE_LOOKING_BACKWARDS: 机头沿着圆周前进的反方向 TOWARDS_HOT_POINT: 机头朝着环绕点 AWAY_FROM_HOT_POINT: 机头反向环绕点 CONTROLLED_BY_REMOTE_CONTROLLER:遥控器控制 USING_INITIAL_HEADING:使用初始角度

2.5.3.2 HotpointMissionOperatorListener

HotpointMissionOperatorListener环绕任务状态监听，包括任务的开始，结束，执行状态反馈

限定符和类型	方法和说明
void	**onExecutionUpdate**(HotpointMissionEvent event) 环绕任务执行实时状态 HotpointMissionEvent环绕任务执行状态
void	**onExecutionStart**() 环绕任务开始执行反馈
void	**onExecutionFinish**(GDUError error) 环绕任务结束反馈

2.5.3.3 HotpointMissionEvent

FollowMeMissionEvent跟随任务执行状态

限定符和类型	方法和说明
FollowMeMissionState	**getCurrentState**() 环绕任务执行状态 HotpointMissionState EXECUTING: 环绕任务执行中
float	**getRadius**() 获取环绕半径，单位 m
float	**getMaxAngularVelocity**() 获取环绕角速度，单位 rad/s, 1e2

2.6、MISC CLASSES

2.6.1 GDUCodecManager

GDUCodecManager用来解码相机视频流，包含H264/H265数据

限定符和类型	方法和说明
构造方法	**GDUCodecManager**(Context context, SurfaceTexture surfaceTexture, int width, int height) 构造解码管理器 surfaceTexture:图像渲染的surface texture view Width：surface的宽 Height:surface的高
void	**sendDataToDecoder**(byte[] data, int len) 发送视频流H264/H265给解码器进行解码
void	**startStoreMp4ToLocal**(String path, String fileName) 开始存储H264/H265视频流为Mp4文件

	到指定路径 path:mp4在手机或者遥控器存储路径 filename: mp4文件名称
void	**stopStoreMp4ToLocal**(String path, String fileName) 停止存储，开始存储后必现调用，不然存储mp4有播放问题
void	**enabledYuvData**(Boolean enable) 开启硬解码yuv数据
void	**setYuvDataCallback**(YuvDataCallback yuvDataCallback) 设置yuv数据的回调监听，必现调用enableYuvData方法，且enable为true
void	**getYuvData**() 获取当前图像的yuv数据，必现调用enableYuvData方法，且enable为true
void	**getRgbaData**() 获取当前图像的RGBA数据，必现调用enableYuvData方法，且enable为true
void	**startStoreMp4ToLocal**(String path, String fileName) 开始存储H264/5图像数据为MP4文件到指定路径 path:mp4存储路径 filename:mp4文件名称
void	**stopStoreMp4ToLocal**() 停止存储H264/5图像数据为MP4文件
void	**storageCurrentStreamToPicture**(String path, String name, CommonCallbacks.CompletionCallback callback) 存储当前视频图像以图片形式到本地 path: 图片路径 name: 图片名称

2.6.2 GDUDiagnostics

GDUDiagnostics飞行器健康管理

限定符和类型	方法和说明
GDUDiagnosticsType	**getType**() 获取异常类型 GDUDiagnosticsType：健康异常类型
int	**getCode**() 异常码
int	**getSubCode**() 异常字码

String	**getReason**() 异常原因
String	**getSolution**() 异常解决方法

2.6.2.1 GDUDiagnosticsType

异常类型

限定符和类型	方法和说明
BATTERY	**电池**
CAMERA	**相机**
FLIGHT_CONTROLLER	**飞控**
GIMBAL	**云台**
REMOTE_CONTROLLER	**遥控器**
RTK	**RTK**
RADAR	**RADAR**

2.6.3 LogManager

日志管理类，获取飞行器上log列表和下载单个飞行器log

2.6.3.1 LogManager

限定符和类型	方法和说明
void	**getAircraftLogList**(OnGetAircraftLogListener<LogItemBean> listener) 获取飞行器日志列表
void	**downloadAircraftLog**(int logId, String logPath, String savePath, OnDownloadLogFileListener listener) 下载飞行器日志文件 **logId**：日志ID； **logPath**：日志在飞机上的相对路径； **savePath**：本地保存完整路径； **listener**：下载进度/结果回调（OnDownloadLogFileListener） | 方法在开始时会先调用 listener.onStart()，下载过程通过 listener.onProgress(long current, long total) 回

	调进度；成功时通过 listener.onSuccess(...)；失败时通过 listener.onFailure(int error)；

2.6.3.2 LogItemBean

LogItemBean类表示一个日志文件项，具有各种属性和方法来处理日志文件信息

限定符和类型	方法和属性	说明
int	**id**	log文件id
String	**fileName**	日志文件名
String	**path**	log文件路径
long	**start**	log开始时间
long	**end**	log结束时间

2.6.4 GeofencingOperator

GeofencingOperator地理围栏管理类，用于管理电子围栏和临时禁飞区（上传、查询、清除）以及状态监听。

限定符和类型	方法和说明
构造方法	**GeofencingOperator**() 创建地理围栏管理对象
void	**addListener**(GeofencingStateListener geofencingStateListener) 添加地理围栏状态监听（电子围栏状态、禁飞区状态）
void	**getVirtualFences**(@Nullable CommonCallbacks.CompletionCallbackWith<VirtualFencePoint> completionCallback) 获取当前电子围栏配置，成功回调VirtualFencePoint
void	**uploadVirtualFences**(VirtualFencePoint points, @Nullable CommonCallbacks.CompletionCallback completionCallback) 上传电子围栏点位；参数非法时返回**GDUError.COMMON_PARAM_ILLEGAL**，超时返回**GDUError.COMMON_TIMEOUT**
void	**closeVirtualFences**(@Nullable CommonCallbacks.CompletionCallback completionCallback) 关闭并清除电子围栏

void	**uploadNoFlyZone**(NoFlyZonePoint points, @Nullable CommonCallbacks.CompletionCallback completionCallback) 上传临时禁飞区；参数非法时返回 **GDUError.COMMON_PARAM_ILLEGAL**，超时返回 **GDUError.COMMON_TIMEOUT**
void	**closeNoFlyZone**(byte id, @Nullable CommonCallbacks.CompletionCallback completionCallback) 按ID关闭并清除指定禁飞区
void	**getNoFlyZoneList**(@Nullable CommonCallbacks.CompletionCallbackWith<List<Byte>> completionCallback) 获取飞机端临时禁飞区ID列表（当前版本预留）

2.6.4.1 GeofencingStateListener

地理围栏状态监听接口。

限定符和类型	方法和说明
void	**onVirtualFenceStateChanged**(VirtualFenceState state) 电子围栏状态变化回调
void	**onNoFlyZoneStateChanged**(NoFlyZoneState state) 临时禁飞区状态变化回调

2.6.4.2 VirtualFencePoint

电子围栏点位对象，用于电子围栏上传和获取。

限定符和类型	方法和说明
int	**getType**() / **setType**(int type) 围栏类型： **0**圆形或扇形（固定3点）， 扇形： 第一个点为圆心点，第二个点为起始边顶点，与圆心构成起始边；第三个点为终止边顶点，与圆心构成终止边；起始边到终止边为顺时针旋转，旋转角度小于360°；半径为起始边长度。 圆形：起始边顶点与终止边顶点相同，半径为起始边长度。 **1**多边形（最多30点）
List<LatLng>	**getPointList**() / **setPointList**(List<LatLng> pointList) 围栏点位列表（至少3点）

2.6.4.3 NoFlyZonePoint

临时禁飞区点位对象，用于禁飞区上传。

限定符和类型	方法和说明
int	**getId**() / **setId**(int id) 禁飞区ID
int	**getType**() / **setType**(int type) 禁飞区类型： **0**圆形或扇形（固定3点）， 扇形： 第一个点为圆心点，第二个点为起始边顶点，与圆心构成起始边；第三个点为终止边顶点，与圆心构成终止边；起始边到终止边为顺时针旋转，旋转角度小于360°；半径为起始边长度。 圆形：起始边顶点与终止边顶点相同，半径为起始边长度。 **1**多边形（最多30点）
List<LatLng>	**getPointList**() / **setPointList**(List<LatLng> pointList) 禁飞区点位列表（至少3点）

2.6.4.4 VirtualFenceState

电子围栏状态枚举，表示电子围栏当前状态（通过**onVirtualFenceStateChanged**回调获取）。

枚举值	值	说明
INSIDE_THE_FENCE	1	围栏内
CLOSE_TO_THE_FENCE	2	靠近围栏
OUTSIDE_THE_FENCE	3	围栏外
FAILURE_OF_THE_FENCE	4	围栏失效（位置无效或切姿态模式失效）
FENCE_CLOSED	5	电子围栏关闭
NEAR_THE_FENCE_NEED_TO_LOCK	6	接近围栏，需上锁/禁飞距离（提示信息）
NEAR_THE_FENCE_NEED_TO_SWITCH	7	接近围栏，需切换姿态距离

限定符和类型	方法和说明
int	**getKey**() 获取当前状态对应的值

VirtualFenceState	**get**(byte key) 根据状态值获取枚举实例；无匹配返回null

2.6.4.5 NoFlyZoneState

临时禁飞区状态枚举，表示禁飞区当前状态（通过**onNoFlyZoneStateChanged**回调获取）。

枚举值	值	说明
INSIDE_NO_FLY_ZONE	1	禁飞区内
CLOSE_TO_NO_FLY_ZONE	2	靠近禁飞区
OUTSIDE_NO_FLY_ZONE	3	禁飞区外
FAILURE_OF_NO_FLY_ZONE	4	禁飞区失效（位置无效或切姿态模式失效）
NO_FLY_ZONE_CLOSED	5	禁飞区关闭
NEAR_NO_FLY_ZONE_NEED_TO_LOCK	6	接近禁飞区，需上锁/禁飞距离（提示信息）
NEAR_NO_FLY_ZONE_NEED_TO_SWITCH	7	接近禁飞区，需切换姿态距离

限定符和类型	方法和说明
int	**getKey**() 获取当前状态对应的值
NoFlyZoneState	**get**(byte key) 根据状态值获取枚举实例；无匹配返回null

2.7 机场开发注意事项

2.7.1 MSDK配置

2.7.1.1 MSDK版本

1. MSDK版本2.0.91及以后版本支持，机场返航和降落
2. 具体使用可参考FlightControllerActivity中的setAircraftOperationScenario方法和startReturnToDock()方法

2.7.1.2 配置飞机为机场模式

1. 调用**GDUFlightController**的**setAircraftOperationScenarioMode**(AircraftOperationScenarioInfo mode, CommonCallbacks.CompletionCallback callback) 方法

2. AircraftOperationScenarioInfo配置：

（1）aircraftMode：飞机模式必须配置为*AIRPORT_MODE*

（2）alternateLandingPointValid：备降点标志必须有效，设置为1

（3）alternateLandingPointLongitude：备降点的经度必须设置

（4）alternateLandingPointLatitude：备降点的纬度必须设置

（5）dockType：机场类型必须设置，根据实际类型设置。

对应关系如下：

dockType	机库
1	K01
2	K02
3	K03
5	K05
21	K02P
31	K03P
51	K05P

2.7.1.3 返航调用方法

1. 机场返航需使用**GDUFlightController**的**startReturnToDock**(CommonCallbacks.CompletionCallback callback) 方法，返航包含2个阶段：返航到Home点和精准降落。
2. 在二维码异常的情况下，飞机会复飞3次，3次失败后会飞到备降点降落。

2.7.1.4 非遥控器场景

MSDK不是运行在遥控器场景下，需要调用GDURemoteController中的setContinueSendRCControlMidValueEnable方法，设置为true，能解决飞机起飞就降落和飞行任务失败的问题

（注：部分内容由豆包工作 AI 生成）